

Machine Learning Based Demand Forecasting and Inventory Optimization for E Commerce:

A Comparative Study of Gradient Boosting and LSTM Models

Devashish Rawat

Department of Computer Science and Engineering

Under the Guidance of Prof. Sanjay Garg

Jaypee University of Engineering and Technology, Guna, India

Abstract

Accurate demand forecasting remains one of the most challenging operational problems in e-commerce. Holding excess inventory locks up working capital, while running short on stock translates directly into lost sales and customer attrition. This study benchmarks four machine learning models, specifically XGBoost, LightGBM, Univariate LSTM, and Multivariate LSTM, against a large retail dataset comprising 913,000 daily sales records spread across 50 products, 10 stores, and a five-year window from 2013 to 2017. A custom feature engineering pipeline converts four raw input fields into 17 predictive features that reflect seasonal cycles, demand autocorrelation, rolling averages, and store and item baselines. Among the models tested, XGBoost produced the strongest results, recording an RMSE of 7.91 and a MAPE of 12.43%, surpassing all LSTM configurations. These predictions were then passed into a classical inventory management framework to derive economic order quantities, safety stock levels, and reorder points for all 50 products using ABC classification, illustrating how machine learning outputs can measurably reduce the risk of both stockouts and overstock situations. A fully deployable interactive dashboard built on Streamlit brings the entire pipeline together, supporting real-time forecasting, side-by-side model evaluation, batch prediction, SHAP-based interpretability, and a practical inventory planning module. The key finding is that for well-structured tabular demand data, carefully engineered gradient boosting consistently outperforms sequence-based deep learning, and that forecasting precision has tangible, quantifiable consequences for inventory management decisions.

Keywords: *demand forecasting, XGBoost, LSTM, LightGBM, feature engineering, inventory management, EOQ, safety stock, SHAP, Streamlit, ecommerce, time series forecasting*

1. Introduction

The rapid expansion of e-commerce has amplified the consequences of inaccurate demand forecasting far beyond what traditional retail environments typically faced. A growing number of stock-keeping units, compressed product life cycles, and demand that shifts sharply with promotions, seasonal trends, and platform-specific dynamics have placed supply chain planners in a persistently difficult position. The core question they face each planning cycle is straightforward in theory but hard in practice: how much of which product needs to be ordered, where should it be positioned, and when should the order go out? Ordering too much freezes capital in slow-moving stock, while ordering too little pushes potential revenue and repeat customers toward competitors.

Traditional forecasting approaches such as ARIMA, Exponential Smoothing, and Holt-Winters carry decades of practical application in retail settings. Their weakness, however, lies in the assumptions of linearity that underpin them; these methods tend to struggle when confronted with the overlapping temporal dynamics, item-level heterogeneity, and multi-frequency seasonality found in modern e-

commerce data. Gradient boosting frameworks (Chen and Guestrin, 2016; Ke et al., 2017) and deep learning sequence models (Hochreiter and Schmidhuber, 1997) have introduced meaningful new capabilities for handling this complexity, yet systematic and rigorous comparisons of these approaches on large-scale multi-item retail datasets are still relatively uncommon in the published literature.

This paper makes four distinct contributions to the field. First, it introduces a structured feature engineering framework tailored to retail time series, generating lag-based, statistical, and behavioral features, and demonstrates that these derived inputs account for the largest portion of predictive accuracy. Second, it conducts a controlled benchmark of four models, specifically XGBoost, LightGBM, Univariate LSTM, and Multivariate LSTM, across 913,000 daily observations, with XGBoost achieving the top performance at an RMSE of 7.91 and a MAPE of 12.43%. Third, it connects machine learning forecasts to a classical inventory optimization framework, deriving economic order quantities, safety stock levels, and reorder points for all 50 products along with sensitivity analysis across varying service levels and lead times. Fourth, it delivers a production-ready Streamlit application enriched with SHAP-based explainability, allowing practitioners to interactively examine individual predictions and their implications for inventory planning.

2. Literature Review

2.1 Statistical Demand Forecasting

Early approaches to demand forecasting were largely built on classical time series methodology. Box and Jenkins (1976) introduced the ARIMA framework, which quickly established itself as the dominant tool for univariate forecasting and was subsequently extended to accommodate seasonality through SARIMA. Taylor (2003) showed that damped trend exponential smoothing could produce reliable accuracy across retail settings. Hyndman and Athanasopoulos (2018) offer a comprehensive treatment of these techniques while also documenting their practical limitations when scaled to multi-item retail environments.

The M competitions (Makridakis et al., 2018, 2020) have systematically tested forecasting methods across thousands of diverse time series over many years, and the recurring finding is that machine learning approaches and ensembles of simpler statistical models tend to outperform standalone deep learning methods, particularly on short and noisy series. The M5 competition (Makridakis et al., 2022), which used Walmart retail sales data with a structure comparable to the dataset examined in this paper, concluded that gradient boosted trees delivered the strongest overall performance.

2.2 Machine Learning for Demand Forecasting

Gradient boosting gained strong traction in retail forecasting after Chen and Guestrin (2016) released XGBoost, with its regularized learning objective and parallelized tree construction. Ke et al. (2017) followed with LightGBM, which adopts leaf-wise tree growth and gradient-based one-side sampling to achieve faster training while maintaining comparable predictive accuracy. Both methods absorb large sets of engineered features without performance degradation, making them a natural choice for structured tabular retail data.

Januschowski et al. (2020) contend that the key advantage machine learning holds in retail settings stems from its capacity to learn relationships across multiple series and to incorporate diverse external signals. Fildes et al. (2022) arrive at a closely related conclusion through a systematic review of the literature, finding that the quality of feature construction has a greater influence on forecast accuracy than the

particular algorithm selected. This observation sits at the heart of the methodology adopted throughout this paper.

2.3 Deep Learning for Time Series

Hochreiter and Schmidhuber (1997) developed the LSTM architecture as a direct solution to the vanishing gradient problem that had long hindered recurrent networks, allowing models to retain and exploit longer-range temporal dependencies. Salinas et al. (2020) extended this line of work with DeepAR, a probabilistic LSTM-based model that demonstrated competitive performance on Amazon’s internal retail forecasting tasks. Despite these advances, the repeated findings from the M competitions suggest that LSTM models do not inherently surpass well-tuned gradient boosting approaches in practical retail forecasting scenarios.

2.4 Inventory Optimization with Forecasting Integration

The theoretical foundations of EOQ, safety stock, and reorder point calculations for continuous review inventory systems are well documented in Silver et al. (2017). Syntetos and Boylan (2005) provide a quantitative treatment of how forecast accuracy, as measured by MAPE, influences safety stock requirements. Huber et al. (2019) demonstrated within a retail context that machine learning based forecasting can directly lower the safety stock needed relative to purely statistical methods. The present study extends this connection to 50 products spread across 10 stores, moving beyond the single-item focus that characterizes much of the earlier work in this space.

2.5 Research Gap

Existing research has examined the relative merits of machine learning versus statistical forecasting methods and has explored the relationship between prediction quality and inventory planning. However, few studies have assembled a complete, end-to-end pipeline that simultaneously incorporates large-scale multi-item and multi-store retail data, disciplined feature engineering supported by ablation analysis, a controlled head-to-head comparison of gradient boosting and LSTM architectures, machine learning driven classical inventory optimization, differentiated inventory strategy through ABC classification, and a live and interpretable production system. This paper addresses that gap directly.

3. Dataset and Exploratory Data Analysis

3.1 Dataset Description

The data source used in this study is the Kaggle Store Item Demand Forecasting Challenge, a publicly available collection of daily sales records drawn from a large online retailer. The dataset contains 913,000 observations covering the period from January 1, 2013 to December 31, 2017, representing 1,826 consecutive days of transactions across 10 stores and 50 distinct items, which together produce 500 unique store-item combinations. The four raw fields present in the dataset are: date (serving as the daily time index), store (an integer identifier from 1 to 10), item (an integer identifier from 1 to 50), and sales (the daily unit sales count, expressed as a non-negative integer). The dataset contains no missing values and no duplicate records.

Table 1. Dataset Characteristics

Attribute	Value
Total Observations	913,000

Attribute	Value
Time Period	January 2013 to December 2017
Duration	1,826 days (5 years)
Number of Stores	10
Number of Items	50
Store Item Combinations	500
Missing Values	0
Duplicate Records	0
Target Variable	sales (daily units sold)
Train and Validation Split	2013 to 2016 for training, 2017 for validation

3.2 Exploratory Data Analysis

3.2.1 Trend and Seasonality

When daily sales are aggregated across all items and stores, a clear upward trajectory is visible over the five-year span, pointing to steady expansion in either customer volume or product uptake across the retailer. Superimposed on this growth trend is pronounced annual seasonality, with demand cycles reproducing reliably from one year to the next. The interplay between the underlying trend and the recurring seasonal pattern confirms that the series is nonstationary, which strengthens the rationale for including temporal and lag-based features as foundational model inputs.

3.2.2 Sales Distribution

The distribution of daily sales values is positively skewed. The bulk of observations fall within the 0 to 100 unit range, which reflects routine day-to-day demand patterns, while a long right tail stretches toward approximately 180 units. These elevated observations most likely correspond to promotional events or seasonal demand spikes rather than random noise in the data.

3.2.3 Store and Item Heterogeneity

Store-level box plots reveal that all 10 stores experience demand spikes, yet each store displays a distinct outlier profile, which suggests that store-specific promotions or local events drive volume surges and that store identity carries meaningful predictive signal. At the item level, Item 15 produces the highest cumulative sales, and there is considerable variation in both average demand and demand volatility across the 50 products. This item-level heterogeneity justifies the inclusion of item-level aggregate features in the model and supports the use of differentiated inventory strategies through ABC classification.

4. Feature Engineering Methodology

In its raw form, the dataset contains only four fields: date, store, item, and sales. The feature engineering pipeline expands these into 17 predictive features grouped into four categories. Processing is performed on data sorted by store, item, and date to maintain the correct temporal ordering within each store-item series, which is a necessary condition for computing lag and rolling window features without introducing data leakage. No feature computed for time period t incorporates any information from t onwards.

4.1 Temporal Features

Six calendar-based features are extracted from the date field: year, month (ranging from 1 to 12), ISO week number (1 to 53), day of month (1 to 31), day of week (0 for Monday through 6 for Sunday), and a binary indicator for weekend days. Together, these features give the model direct representations of within-week, within-month, and year-over-year seasonality cycles. The month and week features carry particular importance in light of the strong annual seasonality pattern identified during exploratory analysis in Section 3.

4.2 Lag Features

Four lag features are constructed by shifting each store-item sales series back by 1, 7, 14, and 28 days respectively. The one-day lag captures short-term autocorrelation in daily demand, while the seven-day lag reflects demand from the same weekday in the previous week. The 14-day and 28-day lags encode biweekly and monthly demand cycles. All lag calculations are performed within individual store-item groups to avoid cross-series leakage, and the first 28 rows of each series are excluded from training because they contain undefined lag values.

4.3 Rolling Statistical Features

Three rolling averages are calculated over windows of 7, 14, and 28 days, each with a one-day lag applied prior to the window to ensure no leakage occurs. This means the rolling mean for day t is computed using only sales from days that preceded t within the window, never incorporating the current day's value. The seven-day rolling mean captures short-term demand momentum, while the 28-day rolling mean serves as a stable monthly demand baseline.

4.4 Aggregate Behavioral Features

Two aggregate features are derived to summarize behavior at the store and item levels across the full training period. The store average sales feature represents the mean daily sales volume for each store taken across all products, while the item average sales feature captures the mean daily sales for each product taken across all stores. These aggregates provide tree-based models with a useful prior estimate of expected demand for any store-item combination, which meaningfully improves the model's ability to generalize across items it has not seen in similar contexts.

Table 2. Feature Engineering Summary

Category	Features	Count	Purpose
Temporal	year, month, week, day, dayofweek, is weekend	6	Seasonality cycles
Lag	sales lag 1, lag 7, lag 14, lag 28	4	Autocorrelation and memory
Rolling Mean	rolling mean 7, rolling mean 14, rolling mean 28	3	Trend and momentum
Aggregate	store avg sales, item avg sales	2	Baseline behavior
Passthrough	store, item	2	Entity identity
Total		17	

5. Modeling Approaches

5.1 Experimental Setup

All four models are trained on data covering 2013 through 2016 and assessed against a full-year holdout comprising 2017 observations. This strict chronological partition produces approximately 730,000 training records and 182,500 validation records. The prediction target is raw daily sales with no log transformation applied to tree-based models. Model performance is assessed on two metrics: RMSE, which applies quadratic penalties to large errors while preserving the original sales unit scale, and MAPE, which offers a dimensionless measure of relative accuracy that is more directly interpretable in a business context.

5.2 XGBoost

XGBoost constructs its ensemble of regression trees sequentially, with each successive tree targeting the residual errors left by the ensemble at that stage. The final configuration uses a squared error objective, 500 estimators, a maximum tree depth of 8, a learning rate of 0.05, and both subsample and column subsampling ratios set to 0.8. A randomized hyperparameter search spanning 25 distinct configurations produced only marginal improvements over these initial settings, which suggests that XGBoost's performance is relatively insensitive to precise hyperparameter values once a well-constructed feature set is in place.

5.3 LightGBM

LightGBM's leaf-wise growth strategy frequently achieves lower training loss with fewer trees compared to XGBoost's level-wise approach. The model is configured in gradient boosting decision tree mode with a regression objective. Hyperparameters were optimized through a randomized search using TimeSeriesSplit cross-validation with three folds and 30 candidate configurations, covering parameters including the number of estimators, learning rate, number of leaves, maximum depth, minimum child samples, subsample fraction, column subsampling ratio, and both L1 and L2 regularization terms. Chronological cross-validation throughout the tuning process ensures no future data leaks into training.

5.4 Univariate LSTM

The Univariate LSTM works directly from raw sales sequences without any engineered features as input. Each store-item series is normalized to the 0 to 1 range using statistics derived exclusively from the training period. The model accepts a 28-day historical window and produces a single one-day-ahead prediction. Its architecture comprises two stacked LSTM layers containing 64 and 32 units respectively, with a dropout rate of 0.2 applied after the first layer, and a sigmoid output unit. Training is conducted using the Adam optimizer with mean squared error loss, a batch size of 64, and early stopping triggered after 10 epochs without validation improvement.

5.5 Multivariate LSTM

The Multivariate LSTM receives all 17 engineered features as input, providing the sequence model access to the same information available to the tree-based approaches. All features and the target variable are jointly normalized using a MultiOutput min-max scaler fitted solely on training data. Input sequences are structured as three-dimensional tensors with shape (samples, 28, number of features). The architecture consists of two stacked LSTM layers of 128 units each, followed by a dropout layer with a rate of 0.2, a fully connected layer with 32 units and ReLU activation, and a linear output unit. All training settings mirror those used for the Univariate LSTM.

6. Experimental Results

6.1 Overall Model Comparison

Table 3 presents the validation results for all four models evaluated on the 2017 holdout period. XGBoost leads with an RMSE of 7.91 and a MAPE of 12.43%, with LightGBM close behind at an RMSE of 7.94 and a MAPE of 12.56%. The LSTM variants lag further back. The Multivariate LSTM achieves an RMSE of 8.37 and a MAPE of 13.11%, while the Univariate LSTM finishes last with an RMSE of 8.77 and a MAPE of 13.91%. The fact that the Multivariate LSTM outpaces its univariate counterpart indicates that access to engineered features does offer a genuine benefit to the sequence model, even if that advantage is not sufficient to close the gap with gradient boosting entirely.

Table 3. Model Performance on 2017 Validation Set

Rank	Model	RMSE	MAPE (%)	Type	Quality Band
1	XGBoost	7.9126	12.43	Gradient Boosted Tree	Very Good (10 to 15%)
2	LightGBM	7.9421	12.56	Gradient Boosted Tree	Very Good (10 to 15%)
3	Multivariate LSTM	8.3659	13.11	Deep Learning	Very Good (10 to 15%)
4	Univariate LSTM	8.7666	13.91	Deep Learning	Very Good (10 to 15%)

All four models fall within the 10 to 15 percent MAPE range, placing them in the Very Good forecast quality category under the Lewis (1982) classification framework. XGBoost's MAPE of 12.43 percent means that, on average, its daily prediction for a given store-item pair deviates by approximately 12.4 percent from the observed sales figure, which corresponds to roughly 8 units per day in absolute terms.

6.2 Feature Importance Analysis

Gain-based feature importance scores from XGBoost indicate that near-term temporal signals exert the greatest influence. The one-day lag contributes the most to prediction accuracy, followed by the seven-day lag and the seven-day rolling mean. This pattern is intuitive: demand is strongly autocorrelated over short time horizons, and these engineered features make that autocorrelation directly accessible to the model's splitting decisions. Calendar features, particularly month and day of week, serve an important secondary function in capturing recurring seasonal patterns. Store and item aggregate features contribute less on an individual basis but likely assist the model in generalizing across the full set of 500 store-item combinations.

6.3 Item Level and Store Level Evaluation

Performance at the item level varies substantially across the 50 products. Items with stable and predictable demand patterns achieve MAPE values below 10 percent, while high-volatility items characterized by intermittent demand spikes produce MAPE values exceeding 18 percent. This variation has direct implications for inventory planning, as products with higher forecast error require proportionally larger safety stock buffers to sustain the target service level. At the store level, however, performance is considerably more uniform, with RMSE values clustering tightly across all 10 stores. This consistency suggests that the store aggregate feature effectively captures store-level baseline demand.

6.4 Why Tree Models Outperform LSTMs Here

Two factors account for the performance advantage held by gradient boosting methods in this setting. First, the feature engineering pipeline explicitly encodes key temporal structures, including demand autocorrelation, weekly seasonality, and trend, which XGBoost and LightGBM can exploit directly through their tree splitting logic without needing to infer these patterns from raw sequential data. Second, LSTM networks rely on backpropagation through time and require substantially more data per learned pattern to function effectively. Within a 28-day input window, the LSTM has limited capacity to independently capture the longer-range patterns that the 28-day rolling mean and 28-day lag features deliver directly to the tree models. The 0.40 RMSE gap between the Multivariate and Univariate LSTM confirms that these engineered features carry genuine predictive value even when supplied to sequence-based models.

7. Inventory Optimization Framework

The demand forecasts produced by XGBoost for the 2017 period are passed into a classical continuous review inventory framework to generate actionable replenishment policies for all 50 products. The framework operates at the individual item level, aggregating forecast demand across all 10 stores to derive system-wide inventory parameters.

7.1 Demand Aggregation

Daily XGBoost predictions are summed across all 10 stores for each product to produce system-level demand statistics. These include total annual forecast demand (D), average daily demand, and the standard deviation of daily demand. The coefficient of variation is also calculated for each item as a measure of demand predictability.

7.2 Economic Order Quantity

The Economic Order Quantity is derived by minimizing the combined annual cost of placing orders and holding inventory. Given a fixed ordering cost S per order, an annual holding cost h per unit, and total annual demand D , the formula takes the following form:

$$EOQ = \text{square root of } (2 \times D \times S / h)$$

The business parameters used throughout this analysis are set as follows: ordering cost S is \$50 per order, holding cost h is \$2 per unit per year, and replenishment lead time L is 7 days. Under these assumptions, the average EOQ computed across all 50 items comes to approximately 194 units per order, with higher-demand products naturally warranting proportionally larger order quantities.

7.3 Safety Stock

Safety stock functions as a buffer to absorb demand variability during the replenishment lead time. At a chosen service level represented by a z -score (1.645 for a 95 percent service level), the safety stock calculation is given by:

$$SS = z \times \text{standard deviation of daily demand} \times \text{square root of } L$$

The direct relationship between demand variability and safety stock means that improvements in forecast accuracy translate immediately into lower inventory carrying costs. As Syntetos and Boylan (2005) demonstrate, a lower MAPE corresponds to a tighter distribution of forecast errors, which in turn reduces the safety stock buffer required at any given service level. The XGBoost model's MAPE of 12.43 percent implies that achieving accuracy below 10 percent would produce a proportional reduction in safety stock

across all 50 products, which represents a concrete and quantifiable financial case for ongoing investment in forecasting quality.

7.4 Reorder Point

The Reorder Point defines the inventory level at which a new purchase order must be placed in order to prevent a stockout before the incoming shipment arrives. It is calculated as follows:

$$ROP = \text{average daily demand} \times L + SS$$

The reorder point accounts for projected demand over the entire lead time period in addition to the safety stock buffer. Products with high demand volumes or high demand variability require higher reorder points, meaning replenishment orders need to be initiated earlier.

7.5 ABC Classification

Products are categorized into three management classes according to their respective share of total annual demand, consistent with the Pareto principle. Class A items, which collectively account for the top 70 percent of cumulative demand, are managed under a stringent 99 percent service level target. Class B items, spanning the 70th to 90th percentile of cumulative demand, operate under a 95 percent service level with moderate replenishment controls. Class C items, which make up the bottom 10 percent of demand, are managed at a 90 percent service level with less frequent, higher-volume orders to reduce ordering overhead. This tiered structure allows inventory management effort and cost to be directed in proportion to the business value of each product group.

Table 4. Inventory Policy Summary (All 50 Items, 95% Service Level)

Metric	Value
Total Annual Forecast Demand	Approximately 2,100,000 units across all items and stores
Average EOQ per Item	Approximately 194 units per order
Average Orders per Year per Item	Approximately 21 orders
Total Safety Stock (all items)	Approximately 6,800 units
Average Reorder Point	Approximately 78 units
Service Level Target	95% ($z = 1.645$)
Lead Time	7 days
Class A Items	Approximately 8 items (top 70% of demand)
Class B Items	Approximately 17 items (next 20% of demand)
Class C Items	Approximately 25 items (bottom 10% of demand)

7.6 Inventory Simulation

A day-by-day continuous review simulation is run to validate the computed inventory parameters under realistic operating conditions. The simulation steps through the entire 2017 period using XGBoost

forecasts for Item 15, which carries the highest demand of any product in the dataset. At each step, outstanding orders are received, inventory is depleted by the day's demand with any shortfall logged as a stockout, and a new order equal to the EOQ is triggered whenever inventory falls to or below the reorder point. Simulation results confirm that the derived reorder point and safety stock levels hold stockout frequency near zero at the targeted 95 percent service level.

8. Production Dashboard System

The complete pipeline, encompassing data processing, model inference, inventory optimization, and SHAP-based explainability, has been deployed as an interactive web application using the Streamlit framework. The live application is accessible at <https://e-commerce-demand-forecasting-mjlqjwbeigqfx8abcq8mua.streamlit.app/> and the full source code is available at <https://github.com/Devashish-Rawat1/DemandSense>.

The dashboard is organized into seven functional modules. The Exploratory Data Analysis module provides interactive Plotly charts covering sales history, seasonality decomposition, and store and item comparisons, each with adjustable filtering controls. The Demand Forecast module enables users to select any combination of model and store-item pair, then renders a time series plot of forecast versus actual sales across the 2017 validation period. The Future Forecast module generates predictions up to 90 days ahead using the Univariate LSTM operating in autoregressive mode, with outputs downloadable as CSV files. The Model Comparison module overlays all four model forecasts on a single time series chart and presents RMSE and MAPE comparisons together with residual analysis plots. The Inventory Planner derives EOQ, safety stock, and reorder points for all products from XGBoost forecasts using business parameters that users can adjust, with results also exportable to CSV. The Batch Predict module accepts user-supplied CSV files containing date, store, and item fields and applies the full feature engineering pipeline to produce predictions at scale. Finally, the SHAP Explainer module uses TreeExplainer from the SHAP library (Lundberg and Lee, 2017) to break individual predictions down into additive feature contributions, presented as waterfall plots and feature importance heatmaps.

9. Discussion

9.1 Key Findings

The primary empirical conclusion of this study is that carefully engineered gradient boosting models consistently outperform LSTM-based deep learning on structured, multi-item retail demand data. This finding aligns with the conclusions of the M5 competition (Makridakis et al., 2022) and with the broader literature, which has increasingly shown that the quality of feature engineering carries more weight than algorithmic sophistication in tabular data settings (Fildes et al., 2022). The RMSE difference between the Univariate LSTM and XGBoost stands at 0.85, representing approximately a 10.7 percent relative improvement. The gap narrows to 0.45 when comparing the Multivariate LSTM against XGBoost, which underscores that the gradient boosting advantage is primarily informational rather than architectural in nature. Tree-based models receive temporal dependency information directly through lag and rolling window features, whereas the Univariate LSTM must attempt to discover these patterns from raw sequential data alone.

9.2 Practical Implications

For practitioners working with multi-item retail demand data, the practical takeaway is clear: gradient boosting supported by systematic feature engineering produces higher forecast accuracy at substantially

lower computational cost and with a simpler deployment footprint than deep learning alternatives. The connection to inventory management makes the business case tangible. Each one-percent reduction in MAPE corresponds to a proportional decrease in safety stock holdings across the full product range, which constitutes a direct and measurable financial return on investment in forecast quality.

9.3 Limitations and Future Work

Several limitations of this work deserve acknowledgment. The dataset does not include exogenous variables such as promotional event flags, public holiday calendars, pricing information, or macroeconomic indicators, all of which are recognized as material demand drivers in retail settings. Adding these signals could plausibly bring MAPE into the 8 to 10 percent range. The LSTM hyperparameter search was also bounded by available compute resources; a more thorough exploration of network depth, dropout schedules, and input sequence lengths may uncover meaningful performance gains for deep learning models. Additionally, the inventory simulation assumes fixed and deterministic lead times, whereas real supply chains are subject to lead time variability that would, in practice, require larger safety stock buffers.

Productive directions for future research include: (1) transformer-based architectures such as Temporal Fusion Transformers (Lim et al., 2021), which have demonstrated strong performance on multi-horizon retail forecasting tasks; (2) probabilistic forecasting methods that quantify prediction uncertainty and enable more theoretically rigorous safety stock calculations; (3) reinforcement learning-based approaches for optimizing inventory control decisions across multiple consecutive periods; and (4) the integration of empirical supplier lead time distributions directly into safety stock derivation.

10. Conclusion

This paper presents a complete, end-to-end demand forecasting and inventory optimization pipeline designed for e-commerce retail environments. Beginning with 913,000 daily sales records, a structured feature engineering pipeline generates 17 predictive features that capture temporal seasonality, short and medium-term demand autocorrelation, and entity-level baseline behavior. A controlled comparison across four machine learning models shows that XGBoost achieves the strongest results, with an RMSE of 7.91 and a MAPE of 12.43 percent on the 2017 holdout set, confirming that gradient boosting applied to engineered features outperforms sequential deep learning for structured tabular retail demand data.

The model forecasts feed directly into a classical inventory optimization framework to compute economic order quantities, safety stock levels, and reorder points for all 50 products, with ABC classification used to apply differentiated service level targets across product tiers. The resulting inventory parameters are validated through a continuous review simulation. The full system is delivered as a production-ready interactive dashboard intended for use by practitioners and engineers, with complete SHAP-based explainability available for individual forecast predictions.

The central lesson drawn from this work is that the principal driver of forecast accuracy in structured retail settings is not the sophistication of the chosen model but the quality of feature engineering applied to the data. Machine learning based demand forecasting integrates naturally with established inventory theory, and the findings presented here contribute to an expanding body of evidence that positions gradient boosting as a well-justified default approach for production retail forecasting applications.

References

Box, G. E. P., and Jenkins, G. M. (1976). Time Series Analysis: Forecasting and Control. Holden Day.

- Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785 to 794.
- Fildes, R., Ma, S., and Kolassa, S. (2022). Retail forecasting: Research and practice. *International Journal of Forecasting*, 38(4), 1283 to 1318.
- Hochreiter, S., and Schmidhuber, J. (1997). Long short term memory. *Neural Computation*, 9(8), 1735 to 1780.
- Huber, J., Gossmann, A., and Stuckenschmidt, H. (2019). Cluster based hierarchical demand forecasting for perishable goods. *Expert Systems with Applications*, 76, 140 to 151.
- Hyndman, R. J., and Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (2nd ed.). OTexts.
- Januschowski, T., Gasthaus, J., Wang, Y., Salinas, D., Flunkert, V., Bohlke Schneider, M., and Callot, L. (2020). Criteria for classifying forecasting methods. *International Journal of Forecasting*, 36(1), 167 to 177.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146 to 3154.
- Lewis, C. D. (1982). *Industrial and Business Forecasting Methods*. Butterworths.
- Lim, B., Arik, S. O., Loeff, N., and Pfister, T. (2021). Temporal fusion transformers for interpretable multi horizon time series forecasting. *International Journal of Forecasting*, 37(4), 1748 to 1764.
- Lundberg, S. M., and Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765 to 4774.
- Makridakis, S., Spiliotis, E., and Assimakopoulos, V. (2018). The M4 Competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4), 802 to 808.
- Makridakis, S., Spiliotis, E., and Assimakopoulos, V. (2020). The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1), 54 to 74.
- Makridakis, S., Spiliotis, E., Assimakopoulos, V., Chen, Z., Gaba, A., Tsetlin, I., and Winkler, R. L. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4), 1346 to 1364.
- Salinas, D., Flunkert, V., Gasthaus, J., and Januschowski, T. (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3), 1181 to 1191.
- Silver, E. A., Pyke, D. F., and Thomas, D. J. (2017). *Inventory and Production Management in Supply Chains* (4th ed.). CRC Press.
- Streamlit Inc. (2023). Streamlit: The fastest way to build and share data apps. <https://streamlit.io>
- Sutskever, I., Vinyals, O., and Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*, 27, 3104 to 3112.
- Syntetos, A. A., and Boylan, J. E. (2005). The accuracy of intermittent demand estimates. *International Journal of Forecasting*, 21(2), 303 to 314.
- Taylor, J. W. (2003). Exponential smoothing with a damped multiplicative trend. *International Journal of Forecasting*, 19(4), 715 to 725.